



Universidad Autónoma del Estado de México
Unidad Académica Profesional Tianguistenco
Ingeniería en Software

Proyecto:

CosmicCatStudio

Manual Técnico

Autores:
Angel Barón García
Abner Alain Gil García
Yorne Alejandrina Santos Bobadilla

Contenido

1. Tabla de ilustraciones	3
2. Introducción	4
3. Descripción general del sistema	5
3.1 Objetivo del software	5
3.2 Arquitectura General del sistema	5
3.3 Tecnologías utilizadas	6
4. Especificaciones del hardware y software	8
4.1 Requisitos mínimos y recomendados del sistema	8
4.2 Configuraciones de software	9
5. Descripción de la arquitectura del sistema	11
5.1 Diagramas de arquitectura	11
6. Instalación	16
6.1 Pasos de Instalación	16
6.2 Instalación de Dependencias	16
6.3 Configuración Inicial	17
7. Configuración del Entorno de Desarrollo	18
7.1 Entornos Locales	18
7.2 Control de Versiones	18
7.3 Estructura de Carpetas	18
8. Módulos y Componentes Funcionales	20
8.1 Descripción de Cada Módulo	20
8.2 Servicios Internos y Externos	20
8.3 Ciclo de Vida de los Datos	20
9. Pruebas	22
9.1 Procedimientos de Prueba	22
9.2 Casos de Prueba	22
9.3 Herramientas de Prueba	23
10. Mantenimiento y Soporte	24
10.1 Actualizaciones	24
10.2 Backups	24

1. Tabla de ilustraciones

<i>Ilustración 1: Diagrama flujo de datos nivel 1</i>	11
<i>Ilustración 2: Diagrama de flujo de datos nivel 2</i>	11
<i>Ilustración 3: Diagrama flujo de datos (Artista) Nivel 3</i>	12
<i>Ilustración 4: Diagrama de flujo de datos (Oyente) Nivel 3</i>	12

2. Introducción

Bienvenido al Manual Técnico de CosmicCatStudio, una plataforma desarrollada para apoyar y promover a artistas emergentes en el ámbito musical. Este documento proporciona a los administradores, desarrolladores y técnicos toda la información necesaria sobre la estructura, configuración y funcionamiento interno de CosmicCatStudio. A diferencia del manual de usuario, aquí se detallan los aspectos técnicos de la plataforma, diseñados para facilitar la implementación, el mantenimiento y el soporte de un sistema que reúne a artistas y oyentes en un entorno colaborativo y creativo.

CosmicCatStudio ofrece funcionalidades avanzadas para la promoción musical, como la edición de perfiles de artistas, la creación y gestión de álbumes, y un sistema de comercio electrónico para la venta de productos exclusivos. Desde la configuración inicial hasta la administración continua, este manual cubre cada aspecto esencial para asegurar que la plataforma funcione de manera óptima y escalable.

En este manual, encontrarás instrucciones detalladas sobre la arquitectura del sistema, configuraciones del servidor, descripción de la base de datos y sus relaciones, y la integración de servicios adicionales, como el módulo de envío de correos y el sistema de autenticación.

Con esta guía, nuestro objetivo es asegurar que la plataforma CosmicCatStudio siga siendo un espacio de encuentro seguro y funcional tanto para artistas como para oyentes, fomentando la creación y exploración musical sin límites. ¡Bienvenidos al universo técnico de CosmicCatStudio!

3. Descripción general del sistema

3.1 Objetivo del software

CosmicCatStudio es una plataforma digital diseñada para conectar a artistas emergentes con su audiencia. El objetivo del software es ofrecer un espacio en el que los artistas puedan compartir su música, construir una comunidad de seguidores y generar ingresos a través de la venta de productos exclusivos. Al mismo tiempo, los oyentes pueden explorar música nueva, apoyar a sus artistas favoritos y disfrutar de una experiencia musical completa y personalizada.

3.2 Arquitectura General del sistema

La arquitectura de CosmicCatStudio está basada en una **arquitectura de microservicios** dentro de un enfoque **cliente-servidor**, organizada en tres capas principales. Este enfoque modular permite que cada componente funcione de manera independiente, lo que facilita el mantenimiento, la escalabilidad y la capacidad de implementar mejoras de forma ágil y efectiva. Cada microservicio es responsable de un conjunto específico de funcionalidades, lo cual permite adaptar o actualizar partes del sistema sin necesidad de realizar cambios extensivos en toda la plataforma.

1. Frontend:

- Es la interfaz de usuario donde los artistas y oyentes interactúan con la plataforma. Está construida con HTML, CSS y JavaScript. Esto permite una experiencia de usuario fluida, en la que las páginas no necesitan recargarse por completo para reflejar cambios, mejorando la rapidez y la usabilidad del sistema.

2. Backend:

- El núcleo funcional del sistema, implementado en PHP, maneja la lógica de la aplicación y las conexiones con la base de datos. En el contexto de una arquitectura de microservicios, el backend está dividido en diferentes servicios especializados, como la gestión de usuarios, el manejo de música y productos, y el sistema de estadísticas. Esto permite que cada microservicio opere de manera autónoma y que las actualizaciones se realicen de forma aislada para cada módulo sin afectar la totalidad del sistema. Además, la plataforma está diseñada para que nuevos servicios puedan integrarse de manera ágil según sea necesario.

3. Base de datos:

- La plataforma utiliza una base de datos MySQL para almacenar de forma estructurada toda la información. Los datos se organizan en

tablas con relaciones clave-foránea para asegurar la integridad y optimizar el rendimiento de las consultas. Las tablas almacenan información de los artistas, oyentes, canciones, productos, historial de reproducción y visitas al perfil, manteniendo una estructura organizada que facilita la escalabilidad de la plataforma en un sistema basado en microservicios.

4. APIs y servicios externos:

- CosmicCatStudio integra servicios externos, como PHPMailer, para el envío de correos electrónicos (utilizado para funciones como la recuperación de contraseñas y notificaciones). La arquitectura de microservicios también permite que cada módulo interactúe a través de una API REST, permitiendo operaciones específicas como la consulta de productos y estadísticas de uso. Esta comunicación mediante APIs facilita la escalabilidad de la plataforma y permite que los módulos se comuniquen entre sí de manera eficaz y sin interdependencias rígidas.

3.3 Tecnologías utilizadas

Para el desarrollo y funcionamiento de CosmicCatStudio, se utilizan las siguientes tecnologías y herramientas:

- **Lenguajes de programación:**
 - **PHP:** Lenguaje backend para el manejo de la lógica de la aplicación y la comunicación con la base de datos.
 - **JavaScript:** Usado en el frontend para la interacción dinámica con el servidor sin recargar la página, mejorando la usabilidad.
 - **HTML y CSS:** Para estructurar y estilizar la interfaz de usuario.
- **Base de datos:**
 - **PHPMyAdmin:** Sistema de gestión de bases de datos que almacena toda la información estructurada de la plataforma, asegurando integridad y relaciones entre entidades como artistas, oyentes y canciones.
- **Librerías y herramientas:**
 - **PHPMailer:** Librería en PHP para el envío de correos electrónicos, esencial para funciones de notificación y recuperación de contraseñas.
 - **JSON:** Formato de intercambio de datos usado para la comunicación entre el frontend y backend.

- **Fetch:** es una API (Application Programming Interface) para realizar solicitudes HTTP en JavaScript, permitiendo a los desarrolladores crear aplicaciones que interactúan con servidores y APIs de terceros.
- **Entorno de desarrollo:**
 - **XAMPP:** Utilizado para el desarrollo y pruebas locales del servidor Apache, MySQL y PHP.
 - **Visual Studio Code:** Editor de código utilizado para el desarrollo y mantenimiento de archivos frontend y backend.

4. Especificaciones del hardware y software

4.1 Requisitos mínimos y recomendados del sistema

1. Servidor:

- **Requisitos mínimos:**

- CPU: Procesador dual-core de 2 GHz o superior
- RAM: 2 GB
- Almacenamiento: 10 GB de espacio disponible en disco
- Conexión a internet: Ancho de banda de al menos 5 Mbps para un rendimiento básico

- **Requisitos recomendados:**

- CPU: Procesador quad-core de 3 GHz o superior
- RAM: 4 GB o más, para soportar múltiples usuarios concurrentes
- Almacenamiento: 20 GB de espacio en disco (considerando espacio adicional para almacenar archivos multimedia como canciones e imágenes en base64)
- Conexión a internet: Ancho de banda de al menos 10 Mbps para un rendimiento óptimo

2. Base de datos:

- **Requisitos mínimos:**

- MySQL con un espacio de almacenamiento mínimo de 2 GB para la base de datos, según el volumen esperado de usuarios y datos.

- **Requisitos recomendados:**

- MySQL con 5 GB de almacenamiento o más, considerando el crecimiento de la base de datos a medida que aumentan las canciones, productos y usuarios.

3. Espacio en disco:

- **Mínimo:** 10 GB de espacio disponible, ideal para una prueba básica de la plataforma.
- **Recomendado:** 20 GB o más para almacenar archivos multimedia (música y fotos en base64), historial de reproducción y estadísticas.

4.2 Configuraciones de software

1. Servidor de desarrollo:

- **XAMPP o alternativa** (por ejemplo, WAMP, LAMP o MAMP):
 - XAMPP proporciona Apache, MySQL y PHP en un entorno de desarrollo local, simplificando la configuración y pruebas del sistema.
- **phpMyAdmin:**
 - Herramienta de administración de MySQL con interfaz gráfica, incluida en XAMPP y otras distribuciones. Facilita la gestión de la base de datos al permitir realizar consultas SQL, administrar tablas y ver datos sin necesidad de usar comandos en línea. Es especialmente útil para tareas de mantenimiento, importación/exportación de datos y configuración de usuarios y permisos.

2. Extensiones y configuraciones necesarias:

- **PHP:**
 - **Versión mínima:** 7.4
 - **Versión recomendada:** 8.0 o superior, para asegurar compatibilidad con las últimas funciones de PHP y mejoras en el rendimiento.
 - **Extensiones necesarias:**
 - **mysqli:** Para la conexión y manipulación de la base de datos MySQL.
 - **Fetch:** utilizado para manejar APIs externas como de terceros.
 - **json:** Para el manejo de datos en formato JSON en la comunicación frontend-backend.
- **MySQL:**
 - **Versión mínima:** 5.7
 - **Versión recomendada:** 8.0 o superior, que incluye mejoras en rendimiento y seguridad.

3. Configuración de correo electrónico:

- **PHPMailer:**

- Librería de PHP para el envío de correos electrónicos. Es utilizada para funciones de notificación y recuperación de contraseñas en CosmicCatStudio.
- **Requisitos de configuración:**
 - Servidor SMTP: Especificar el servidor SMTP para el envío de correos (por ejemplo, smtp.gmail.com para pruebas con Gmail).
 - Puerto: Normalmente 587 para conexiones TLS o 465 para conexiones SSL.
 - Autenticación: Configuración de credenciales SMTP (usuario y contraseña).
 - Encriptación: Configuración recomendada de seguridad TLS o SSL.

4. Otros softwares recomendados para el entorno de desarrollo:

- **Editor de código:** Visual Studio Code o cualquier otro editor que permita una gestión eficiente de proyectos de PHP y HTML.
- **Control de versiones:** Git, para mantener versiones del código y facilitar la colaboración entre desarrolladores.

5. Descripción de la arquitectura del sistema

5.1 Diagramas de arquitectura

1. Diagrama de flujo de datos:

- Este diagrama muestra cómo se mueven los datos dentro del sistema *CosmicCatStudio*, desde la entrada de información por parte de los usuarios hasta el almacenamiento en la base de datos y la generación de resultados. Incluye entidades como usuarios (artistas y oyentes), la base de datos y los procesos de autenticación, gestión de música y compras.

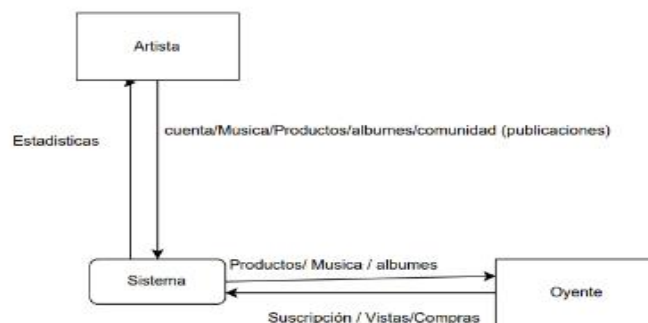


Ilustración 1: Diagrama flujo de datos nivel 1

- Como se muestra en la *ilustración 1*, este diagrama ilustra el flujo de información entre los usuarios (artistas y oyentes) y el sistema. Los artistas interactúan con el sistema para gestionar cuentas, subir música, productos y publicaciones, mientras que los oyentes acceden a productos y música, realizan compras, y visualizan contenido. El sistema también devuelve estadísticas a los artistas.

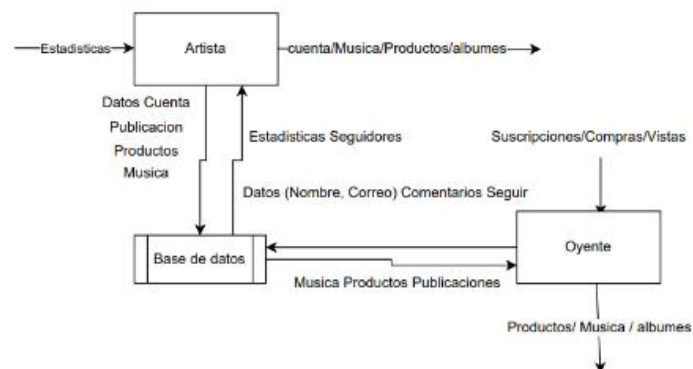


Ilustración 2: Diagrama de flujo de datos nivel 2

- Como se muestra en la *ilustración 2*, este diagrama expande el flujo de datos, mostrando cómo los datos de artistas y oyentes se almacenan y se gestionan en la base de datos. Los artistas pueden acceder a sus estadísticas de seguidores y gestionar su contenido, mientras que los oyentes pueden suscribirse, comprar y seguir a los artistas. La base de datos centraliza y distribuye la información de publicaciones, música y productos entre artistas y oyentes.

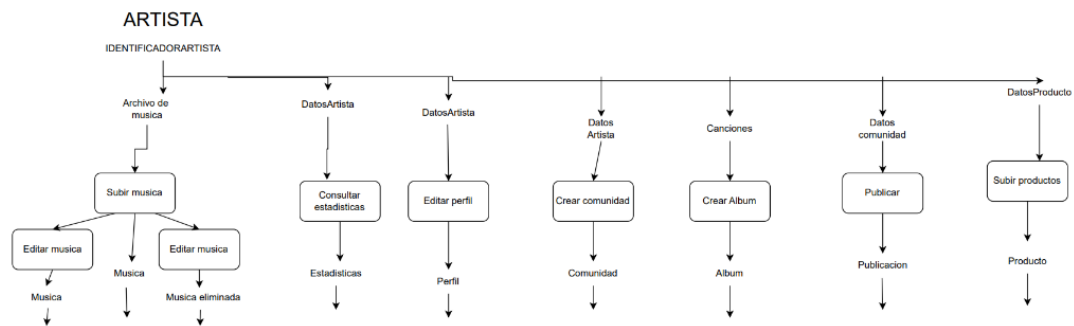


Ilustración 3: Diagrama flujo de datos (Artista) Nivel 3

- Como se muestra en la ilustración 3, este diagrama detalla los procesos específicos que los artistas pueden realizar en la plataforma, como subir y editar música, consultar estadísticas, gestionar su perfil, crear una comunidad, publicar álbumes y ofrecer productos. Cada acción está vinculada a los datos específicos de la base de datos (como música, estadísticas, comunidad, etc.), permitiendo a los artistas interactuar y gestionar su contenido.

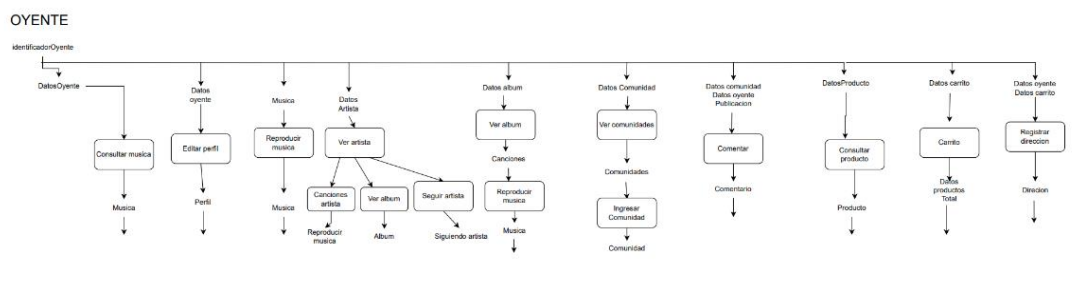


Ilustración 4: Diagrama de flujo de datos (Oyente) Nivel 3

- Este diagrama muestra las acciones disponibles para los oyentes, como consultar música, editar su perfil, reproducir canciones, seguir a artistas, unirse a comunidades y comprar productos. Cada proceso está vinculado a datos específicos del oyente, permitiéndoles interactuar con la plataforma, explorar contenido y realizar compras.

2. Diagrama de flujo de procesos:

- Este diagrama describe los procesos principales de *CosmicCatStudio* y cómo se desarrollan en pasos secuenciales. Muestra el flujo desde la interacción inicial del usuario (como registro o autenticación) hasta operaciones más complejas como subir música, gestionar perfiles, y realizar compras.

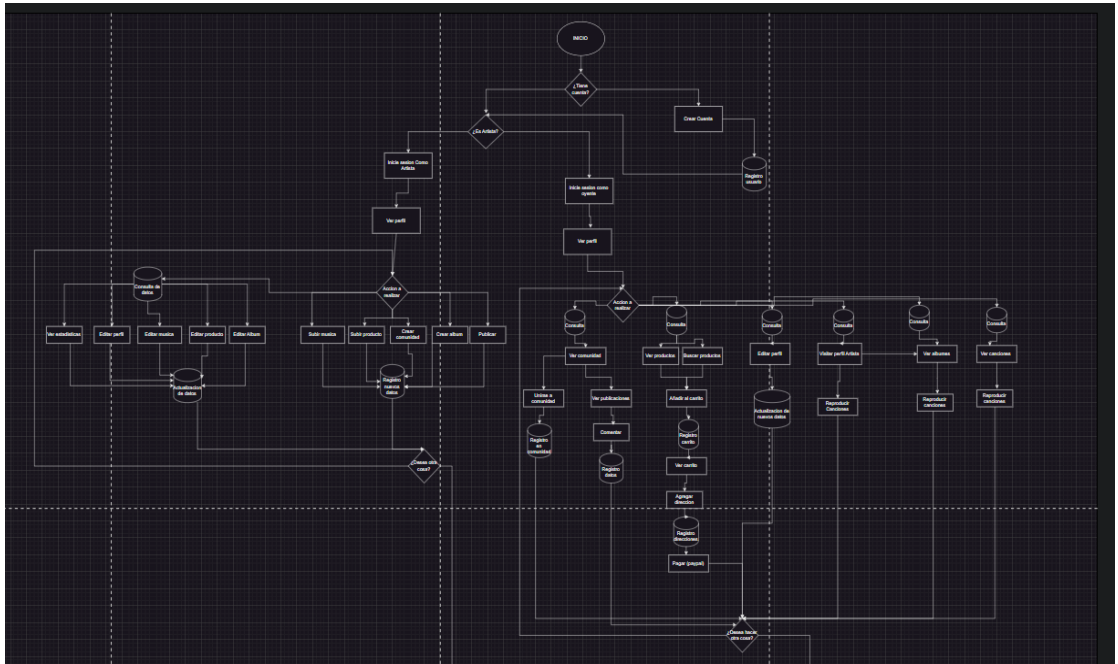


Ilustración 5: Diagrama de flujo de procesos

Este diagrama de flujo de procesos muestra el recorrido principal de los usuarios en la plataforma *CosmicCatStudio*, tanto para artistas como para oyentes. Los usuarios inician sesión o crean una cuenta, y luego se diferencian en dos caminos según el tipo de cuenta:

1. Para los artistas:

- Los artistas pueden ver su perfil y acceder a varias funciones, como editar perfil, subir música, gestionar productos, crear álbumes, y administrar su comunidad. También tienen acceso a sus estadísticas y pueden realizar publicaciones en la comunidad.

2. Para los oyentes:

- Los oyentes pueden explorar el perfil de artistas, ver productos, álbumes y canciones, y añadir productos al carrito para realizar compras. También tienen la opción de editar su perfil, interactuar con publicaciones de la comunidad y gestionar sus preferencias en el carrito de compras.

3. Diagrama de componentes:

- El diagrama de componentes representa la estructura física del sistema, incluyendo el frontend, backend, y base de datos, así como la interacción con servicios externos como PHPMailer. Este diagrama muestra cómo están organizados los módulos del sistema, especificando los componentes clave como el servidor web, los microservicios, la API y la conexión con la base de datos.

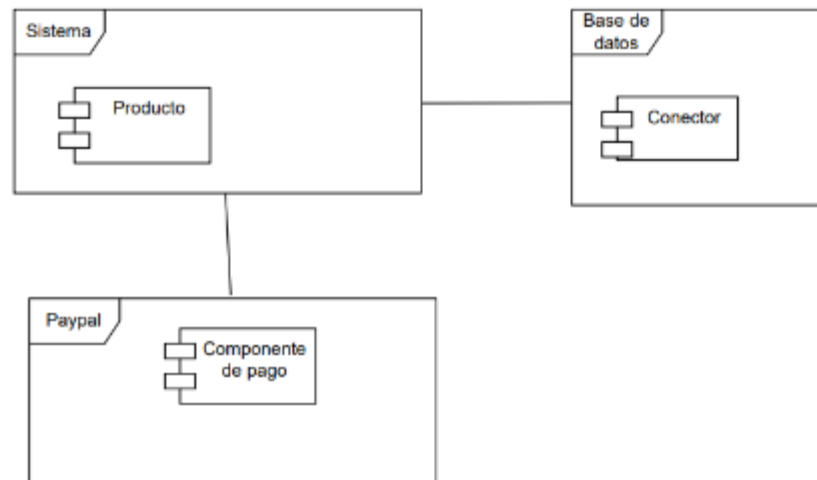


Ilustración 6: Diagrama de componentes

Este diagrama de componentes muestra la relación entre los elementos clave del sistema de *CosmicCatStudio* involucrados en el proceso de compra de productos.

1. **Sistema:**

- El sistema contiene el módulo de **Producto**, encargado de gestionar la información y disponibilidad de los productos que los artistas ofrecen para la venta en la plataforma.

2. **Base de datos:**

- La **Base de datos** contiene el **Conector**, que es el componente responsable de conectar el sistema con la base de datos, permitiendo el acceso y manejo de la información sobre productos, usuarios y transacciones.

3. **PayPal:**

- La integración con **PayPal** incluye un **Componente de pago** que facilita el procesamiento de pagos. Este componente permite a los usuarios realizar transacciones de manera segura, conectando la

plataforma con el sistema de pagos de PayPal para completar la compra de productos.

- Este diagrama permite a los técnicos y desarrolladores tener una visión clara de los elementos de hardware y software involucrados, así como sus interconexiones.

6. Instalación

6.1 Pasos de Instalación

1. Instalación de Servidor y Software Requerido:

- **XAMPP:** Instale XAMPP en el equipo, que incluye Apache (para el servidor web) y MySQL (para la base de datos).
- **PHP y phpMyAdmin:** Asegúrese de que estén correctamente instalados dentro de XAMPP
- **Editor de Código:** Descargue Visual Studio Code, junto con la extensión *PHP Server Extension* si quiere trabajar sin XAMPP en el futuro.

2. Configuración del Servidor:

- Inicie Apache y MySQL desde el panel de control de XAMPP.
- Verifique que el servidor funcione correctamente accediendo a <http://localhost> en el navegador.

3. Configuración del archivo my.ini:

- Abra el archivo my.ini (ubicado en la carpeta de instalación de MySQL en XAMPP) y realice los ajustes necesarios:
 - **Puertos:** Confirme que el puerto de MySQL esté configurado correctamente, port=3306.
 - **Límite de tamaño para cargas de archivos:** Ajuste el valor de max_allowed_packet (ej. max_allowed_packet=16M).
 - Guarde los cambios y reinicia MySQL desde XAMPP para aplicar los ajustes.

6.2 Instalación de Dependencias

1. PHPMailer:

- Asegúrese que el proyecto esté incluido en la carpeta PHPMailer.
- Tiene que configurar PHPMailer en el archivo PHP que envía correos, definiendo el servidor SMTP y credenciales según tu proveedor de correos.

2. Importación de la Base de Datos:

- Abra phpMyAdmin desde <http://localhost/phpmyadmin>.
- Crea una nueva base de datos llamada cosmiccatstudio.

- Utilice la opción de "Importar" en phpMyAdmin para cargar el archivo SQL que contiene las 20 tablas del proyecto.
- Verifique que las tablas se hayan importado correctamente y que las relaciones y datos iniciales estén bien configurados.

6.3 Configuración Inicial

1. Permisos de Usuarios:

- Desde phpMyAdmin, configure los permisos del usuario root o cree usuarios específicos si desea más seguridad, asignándoles permisos según las necesidades del proyecto.

2. Variables de Entorno:

- Configure las variables de entorno para aspectos críticos como las credenciales de base de datos, directorios de recursos (imágenes, música, etc.) y credenciales para el servidor SMTP de correos.

3. Pruebas de Conexión:

- Compruebe que los archivos PHP puedan conectarse a MySQL sin problemas, verificando que las credenciales en el código coincidan con los del servidor.
- Realice pruebas iniciales de carga en las páginas principal.html, subir_musica.php, y otras para confirmar que todos los componentes funcionen como se espera.

7. Configuración del Entorno de Desarrollo

7.1 Entornos Locales

1. Configuración de XAMPP:

- Asegúrese de tener XAMPP porque es esencial para ejecutar Apache y MySQL localmente.
- **Iniciar Servicios:** Active Apache y MySQL desde el panel de XAMPP para que tu aplicación web esté accesible en `http://localhost`.
- **phpMyAdmin:** Acceda a phpMyAdmin para gestionar la base de datos, crear nuevas tablas y ejecutar consultas.

2. PHP Server Extension en Visual Studio Code:

- Si trabaja sin XAMPP, Visual Studio Code tiene la extensión PHP Server Extension.
- **Configuración:** Instale la extensión y active el servidor PHP desde el panel de VS Code. Asegúrese de establecer el directorio raíz correctamente para que sirva archivos PHP sin problemas.
- **Pruebas de Conexión:** Pruebe rutas específicas para verificar que la base de datos y otros recursos funcionen como en XAMPP.

7.2 Control de Versiones

1. Uso de Git:

- **Inicialización del Repositorio:** Inicia el repositorio en el directorio raíz del proyecto con `git init`.
- **Commit de Cambios:** Usa `git add` y `git commit` regularmente para gestionar versiones de tu proyecto.
- **Ramas (Branches):** Trabaja con ramas separadas para el desarrollo de funcionalidades específicas, y luego haz *merge* a la rama principal.

7.3 Estructura de Carpetas

• Frontend (HTML/CSS/JS):

- Ubicado en la carpeta principal del proyecto.
- Incluye todos los archivos `.html` y los recursos asociados como CSS y JavaScript para la interfaz del usuario.

• Backend (PHP):

- En las carpetas php seguido de un número, se encuentran los archivos PHP que contienen la lógica del servidor, como el procesamiento de formularios y la conexión con la base de datos.
- **Recursos Multimedia:**
 - Imágenes, archivos de música, y otros recursos multimedia se almacenan en carpetas específicas (img). Asegúrese de que estas carpetas tengan los permisos necesarios para permitir la carga y descarga de archivos.
- **Bibliotecas Externas:**
 - La biblioteca PHPMailer y cualquier otra dependencia externa se almacenan en carpetas dedicadas para facilitar la gestión.

8. Módulos y Componentes Funcionales

8.1 Descripción de Cada Módulo

1. Flujo de Usuario:

- **Artistas:**
 - Los artistas pueden crear una cuenta, cargar canciones, subir productos para vender, actualizar sus perfiles y ver estadísticas de su perfil y reproducciones.
 - Pueden ver y gestionar su comunidad, interactuar a través de publicaciones y monitorear el seguimiento de sus oyentes.
- **Oyentes:**
 - Los oyentes pueden explorar artistas, escuchar canciones, gestionar su perfil, participar en la comunidad y hacer compras de artículos de sus artistas favoritos.
 - Cuentan con un historial de escuchas recientes y una lista de reproducción personalizada.

8.2 Servicios Internos y Externos

1. APIs y Servicios Internos:

- **API de Base de Datos:** Permite la comunicación entre la base de datos y el frontend para obtener y actualizar datos (por ejemplo, para cargar canciones, obtener perfiles de artistas, etc.).
- **PHPMailer:** Utilizado para el envío de correos electrónicos, como la recuperación de contraseñas. Configurado con un servidor SMTP específico en los archivos de configuración de PHP.

8.3 Ciclo de Vida de los Datos

1. Creación de Datos:

- Los datos de usuarios (oyentes y artistas) se crean en el registro y se almacenan en sus respectivas tablas (oyente y artistas).
- Los productos y canciones se crean en formularios de subida y se almacenan en las tablas producto y musica, respectivamente.

2. Actualización de Datos:

- Los perfiles de oyentes y artistas pueden actualizarse en cualquier momento mediante formularios de edición, que ejecutan consultas de actualización sobre las tablas correspondientes.

- La cantidad de reproducciones, seguidores, y otras estadísticas se actualizan automáticamente en función de la actividad del usuario.

3. Eliminación de Datos:

- Los datos como publicaciones de la comunidad, canciones o productos pueden eliminarse desde el perfil del usuario.
- La eliminación de un registro también puede desencadenar la eliminación de otros datos relacionados (por ejemplo, borrar una canción podría eliminar su entrada en el historial de escuchas de los oyentes).

9. Pruebas

9.1 Procedimientos de Prueba

1. Prueba de Funcionalidad por Módulo:

- **Registro de Usuarios:** Verifique que los usuarios (oyentes y artistas) puedan registrarse exitosamente. Asegúrese de que el correo electrónico y la contraseña se validen correctamente y que se envíe un correo de confirmación cuando aplique.
- **Reproducción de Música:** Confirme que las canciones se carguen correctamente, puedan reproducirse, pausarse, y avanzarse o retroceder en la línea de tiempo.
- **Compras en la Tienda:** Pruebe la funcionalidad completa del carrito de compras, incluyendo la adición y eliminación de productos, y el proceso de pago.
- **Gestión de Perfil y Publicaciones:** Valide que los usuarios puedan editar sus perfiles, crear publicaciones y cargar imágenes en sus comunidades.

9.2 Casos de Prueba

1. Registro de Usuarios:

- **Caso de prueba:** Un nuevo oyente se registra con un correo electrónico y una contraseña válidos.
 - **Expectativa:** El oyente debería ver un mensaje de confirmación.
- **Caso de prueba:** Un artista intenta registrarse con un correo ya registrado.
 - **Expectativa:** El sistema debería mostrar un error de duplicado.

2. Reproducción de Música:

- **Caso de prueba:** El oyente selecciona una canción para reproducirla.
 - **Expectativa:** La canción debería empezar a reproducirse sin interrupciones y reflejarse en el historial de escuchas.
- **Caso de prueba:** El oyente intenta avanzar a la siguiente canción.
 - **Expectativa:** La canción debería avanzar a la siguiente canción sin problemas.

3. Compras:

- **Caso de prueba:** Un oyente añade un producto al carrito y procede a la compra.
 - **Expectativa:** La transacción debería completarse y el inventario actualizarse correctamente.
- **Caso de prueba:** El oyente intenta comprar un producto sin stock.
 - **Expectativa:** El sistema debería mostrar un mensaje de error.

9.3 Herramientas de Prueba

1. Unit Testing:

- Utilice un framework de pruebas unitarias en PHP, como PHPUnit, para probar funciones específicas, especialmente aquellas que manejan lógica crítica en el backend, como el registro y la autenticación de usuarios.

2. Pruebas de Carga:

- Para probar la capacidad de respuesta y escalabilidad, utiliza herramientas como Apache JMeter o locust.io para simular múltiples usuarios accediendo al sistema simultáneamente.

3. Pruebas de Integración:

- Pruebe los flujos completos que incluyen múltiples módulos (por ejemplo, un usuario que se registra, accede a la tienda y realiza una compra) para asegurar que todo el sistema interactúe correctamente.

10. Mantenimiento y Soporte

10.1 Actualizaciones

1. Procedimiento para Actualizar el Sistema:

- **Verificación de Cambios:** Antes de aplicar una actualización, revise los cambios en un entorno de pruebas (staging) para asegurar que las nuevas funciones o correcciones de errores no afecten la funcionalidad actual.
- **Respaldo Previo:** Realizar un backup completo de la base de datos y de los archivos del sistema antes de proceder.
- **Despliegue de la Actualización:**
 - Suba los archivos modificados al servidor o realiza la actualización del sistema.
 - Realizar una prueba rápida de las funcionalidades principales (inicio de sesión, reproducción de música, compras, etc.) para confirmar que la actualización fue exitosa.
- **Documentación:** Actualizar la documentación para reflejar los nuevos cambios, incluyendo pasos de instalación si hay nuevas dependencias.

10.2 Backups

1. Estrategias de Respaldo:

- **Backups Automáticos Diarios:** Programar copias de seguridad diarias de la base de datos usando scripts de cron (o similar) que exporten la base de datos a un archivo .sql.
- **Backups Semanales del Sistema Completo:** Realizar un respaldo completo de los archivos del servidor semanalmente, incluyendo las carpetas de multimedia y los archivos PHP.

2. Restauración de la Base de Datos:

- **Desde phpMyAdmin:** En caso de pérdida de datos, usa phpMyAdmin para importar el archivo .sql de respaldo. Asegúrese de que el archivo no exceda los límites de tamaño configurados en my.ini.